OneCompiler

Pricing Learn Code Deploy More Login

HelloWorld.cpp

hashing using Linear Probing method

AI CPP RUN

```c
#include <stdio.h>
#define SIZE 10

int hashTable[SIZE];

void insert(int key) {
    int index = key % SIZE;

    while (hashTable[index] != -1) {
        index = (index + 1) % SIZE;
    }
    hashTable[index] = key;
}

void display() {
    for (int i = 0; i < SIZE; i++) {
        printf("%d -> %d\n", i, hashTable[i]);
    }
}

int main() {
    for (int i = 0; i < SIZE; i++)
        hashTable[i] = -1;

    insert(23);
    insert(43);
    insert(13);

    display();
    return 0;
```

STDIN

Input for the program (Optional)

Output:

2 ms | 4.4 MB

```
0 -> -1
1 -> -1
2 -> -1
3 -> 23
4 -> 43
5 -> 13
6 -> -1
7 -> -1
8 -> -1
9 -> -1
```

OneCompiler

Pricing Learn Code Deploy More Login

HelloWorld.cpp

arrange a series of numbers using Insertion Sort

AI CPP RUN

```c
#include <stdio.h>

int main() {
    int arr[100], n, i, j, key;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    for(i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        while(j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }

    printf("Sorted array:\n");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

STDIN

1 2 3 4 5

Output: 2 ms | 4.7MB

```
Enter number of elements: Enter elements:
Sorted array:
2
```

OneCompiler

Pricing Learn Code Deploy More Login

HelloWorld.cpp

Merge Sort

AI CPP RUN

```
    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;

        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = 6;

    mergeSort(arr, 0, n - 1);

    printf("Sorted array:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

STDIN

Input for the program (Optional)

Output: 2 ms | 4.0MB

```
Sorted array:
5 6 7 11 12 13
```

OneCompiler

Pricing Learn Code Deploy More Login

HelloWorld.cpp

Quick Sort

AI CPP RUN

```
    }

    temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = 6;

    quickSort(arr, 0, n - 1);

    printf("Sorted array:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

STDIN

Input for the program (Optional)

Output: 2 ms | 4.7 MB

```
Sorted array:
1 5 7 8 9 10
```

OneCompiler

Pricing Learn Code Deploy More Login

HelloWorld.cpp

Heap Sort

AI CPP RUN

```cpp
#include <stdio.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    int temp;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i >= 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
```

STDIN

Input for the program (Optional)

Output: 2 ms | 4.5 MB

```
Sorted array:
1 3 4 5 10
```

OneCompiler

Pricing Learn Code Deploy More Login

HelloWorld.cpp

AVL Tree (Insert, Delete, Search)

AI CPP RUN

```
        return search(root->left, key);

    return search(root->right, key);
}

void inorder(struct Node *root) {
    if(root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}

int main() {
    struct Node *root = NULL;

    root = insert(root, 10);
    root = insert(root, 20);
    root = insert(root, 30);

    printf("Inorder traversal: ");
    inorder(root);

    if (search(root, 20))
        printf("\nFound");
    else
        printf("\nNot Found");

    return 0;
}
```

STDIN

Input for the program (Optional)

Output: 2 ms | 4.4 MB

```
Inorder traversal: 10 20 30
Found
```

OneCompiler

Pricing Learn Code Deploy More Login

HelloWorld.cpp

Breadth First Search (BFS)

AI CPP RUN

```cpp
#include <stdio.h>
#define MAX 10

int queue[MAX], front = -1, rear = -1;
int visited[MAX];

void enqueue(int v) {
    if (rear == MAX - 1) return;
    if (front == -1) front = 0;
    queue[++rear] = v;
}

int dequeue() {
    return queue[front++];
}

void BFS(int graph[MAX][MAX], int n, int start) {
    enqueue(start);
    visited[start] = 1;

    while (front <= rear) {
        int v = dequeue();
        printf("%d ", v);

        for (int i = 0; i < n; i++) {
            if (graph[v][i] == 1 && !visited[i]) {
                enqueue(i);
                visited[i] = 1;
            }
        }
```

STDIN

Input for the program (Optional)

Output: 2 ms | 4.5 MB

```
0 1 2 3 4
```

OneCompiler

Pricing Learn Code Deploy More Login

HelloWorld.cpp

Depth First Search (DFS)

AI CPP RUN

```c
#include <stdio.h>
#define MAX 10

int visited[MAX];

void DFS(int graph[MAX][MAX], int n, int v) {
    printf("%d ", v);
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (graph[v][i] == 1 && !visited[i]) {
            DFS(graph, n, i);
        }
    }
}

int main() {
    int n = 5;
    int graph[MAX][MAX] = {
        {0,1,1,0,0},
        {1,0,1,1,0},
        {1,1,0,0,1},
        {0,1,0,0,1},
        {0,0,1,1,0}
    };

    DFS(graph, n, 0);
    return 0;
}
```

STDIN

Input for the program (Optional)

Output: 2 ms | 4.5 MB

```
0 1 2 4 3
```

OneCompiler

Pricing Learn Code Deploy More Login

HelloWorld.cpp

Dijkstra's Algorithm (Shortest Path)

AI CPP RUN

```cpp
#include <stdio.h>
#define MAX 10
#define INF 9999

int minDistance(int dist[], int visited[], int n) {
    int min = INF, min_index;

    for (int i = 0; i < n; i++) {
        if (!visited[i] && dist[i] <= min) {
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}

void dijkstra(int graph[MAX][MAX], int n, int src) {
    int dist[MAX], visited[MAX];

    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {
        int u = minDistance(dist, visited, n);
        visited[u] = 1;

```

STDIN

Input for the program (Optional)

Output: 2 ms | 4.4 MB

```
Vertex   Distance from Source
0    0
1    7
2    8
3    8
4    5
```

OneCompiler

Pricing Learn Code Deploy More Login

HelloWorld.cpp

Prim's Algorithm (Minimum Spanning Tree)

AI CPP RUN

```cpp
#include <stdio.h>
#define MAX 10
#define INF 9999

int minKey(int key[], int mstSet[], int n) {
    int min = INF, min_index;

    for (int v = 0; v < n; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

void primMST(int graph[MAX][MAX], int n) {
    int parent[MAX];
    int key[MAX];
    int mstSet[MAX];

    for (int i = 0; i < n; i++) {
        key[i] = INF;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < n - 1; count++) {
```

STDIN

Input for the program (Optional)

Output: 2 ms | 4.5 MB

```
Edge   Weight
0 - 1    2
1 - 2    3
0 - 3    6
1 - 4    5
```

OneCompiler

Pricing Learn Code Deploy More Login

HelloWorld.cpp

Kruskal's Algorithm (Minimum Spanning Tree)

AI CPP RUN

```cpp
#include <stdio.h>
#include <stdlib.h>

struct Edge {
    int u, v, w;
};

int parent[10];

int find(int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int u, int v) {
    int a = find(u);
    int b = find(v);
    parent[a] = b;
}

void kruskal(struct Edge edges[], int n, int e) {
    int count = 0, i = 0, cost = 0;

    while (count < n - 1 && i < e) {
        int u = edges[i].u;
        int v = edges[i].v;
        int w = edges[i].w;

        if (find(u) != find(v)) {
```

STDIN

Input for the program (Optional)

Output: 1 ms | 4.5 MB

```
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Total cost: 19
```